PortScan-OS

Port Scanning with OS Information and Version Detection

A Custom Attack Toolkit and Defensive Countermeasure Architecture from Scratch

Assigned Topic: 8
Topic Title: Port Scanning with OS information / version
Subsection: A2
Supervisor: **Anwarul Bashir Shuaib**
Submitted By: **Nawriz Ahmed Turjo (2105032)**
Abhishek Roy (2105033)

Core Attack Pipeline & Defensive Countermeasures

Raw Layer-2 TCP SYN Port Scanner

Stream Banner Grabbing (FTP & HTTP)

Heuristic TCP/IP OS Fingerprinting

Sliding-Window Dynamic Firewall Defense

1 Overview of the Project Idea

1.1 Problem Statement and Threat Model

Network reconnaissance determines which communication channels and service ports are reachable across a network perimeter, what software daemon versions run on those ports, and what operating system kernel manages the host. While off-the-shelf tools like Nmap automate this process, building a custom reconnaissance engine demonstrates how raw packets are serialized and parsed on Windows. Operating within an adjacent local subnet, this toolkit manually constructs Ethernet, IPv4, and TCP frames to identify reachable ports, fingerprint the host operating system, and validate exposed services within defined safety limits.

1.2 Project Scope and Five-Phase Attack Chain

The project organizes its workflow into five attack phases alongside a concurrent defensive module:

Table 1: Five-Phase Reconnaissance and Attack Architecture

Phase	Designation	Operational Focus	Engineering Mechanism
1	**TCP SYN Port Scan**	Detect Open, Closed, and Filtered ports	Raw Layer-2 frame construction via Npcap; half-open scan without completing the 3-way handshake.
2	**Banner Grabbing**	Service and version identification	Standard stream sockets querying listening daemons (passive capture for FTP, active HTTP probes).
3	**OS Fingerprinting**	Identify target host operating system	Dual-candidate heuristic scoring of SYN/ACK and RST headers: TTL, window size, RFC 6864 IP ID, and option ordering.
4	**Vulnerability Match**	Map findings to attack surfaces	Automated mapping between extracted daemon versions and known proof-of-concept exploit vectors.
5	**Exploit Execution**	Demonstrate concrete security compromise	Automated dispatch of anonymous FTP authentication and HTTP directory path traversal payloads.
Defense	**Scan Detection & Block**	Passive detection and dynamic mitigation	Sliding-window inspection tracking SYN packet rates and port diversity with dynamic Windows Firewall mitigation.

1.3 Core Design Principles

The attack and defense modules adhere to four core architectural constraints:

- **From-Scratch Packet Assembly:** Serializes raw Ethernet II, IPv4, and TCP headers byte by byte in binary format without high-level packet construction libraries.
- **RFC 1071 Checksum Computation:** Computes 16-bit one's complement checksums over IPv4 headers and TCP pseudo-headers using a custom mathematical routine.
- **Npcap Low-Level Driver Interface:** Uses Npcap via Python `ctypes` strictly for link-layer frame transmission and promiscuous capture on Windows.
- **Integrated Host Defense:** Pairs offensive reconnaissance with a concurrent monitor that detects active scanning and dynamically injects firewall quarantine rules.

1.4 Safety Architecture and Operational Safeguards

To maintain ethical standards and prevent disruption in laboratory networks, the toolkit enforces five non-bypassable safeguards:

1. **RFC 1918 Scope Validation:** Targets must belong to RFC 1918 private subnets (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`); external destinations are rejected immediately.
2. **Bounded Probe Horizon:** Caps probe lists at 50 ports per run (MAX_PROBES $= 50$), prohibiting full 65,535-port scans and subnet sweeps.
3. **Mandatory Rate Limiting:** Enforces an inter-packet delay $\geq 0.2\,\text{s}$ (default: $0.5\,\text{s}$) to avoid buffer saturation or switch broadcast storms.
4. **Source Port and Sequence Randomization:** Rotates ephemeral source ports with randomized initial sequence numbers to prevent state collision.
5. **Non-Destructive Read-Only Exploits:** Restricts exploit modules to read-only checks (FTP `PWD`/`LIST` and HTTP static file traversal).

2 Network Topology and System Components

2.1 Laboratory Network Architecture

The experimental testbed operates within an isolated private local subnet (`192.168.0.0/24`) on a dedicated physical switch. All nodes share the same Layer-2 broadcast domain because the custom scanner crafts raw Ethernet frames and issues direct Address Resolution Protocol (ARP) queries to resolve target hardware addresses.

To evaluate OS fingerprinting across diverse TCP/IP implementations, the testbed incorporates four physical target platforms representing major operating system families, as detailed in Table 2.

Table 2: Heterogeneous Target Device Specifications in the Laboratory Testbed

Device Role	Operating System Stack	Assigned IPv4	Active Services & Verification Role
Attacker Station	Windows 11 Pro (x86_64)	`192.168.0.105`	Toolkit orchestrator, raw Npcap packet injection, and capture monitoring.
Primary Victim	Windows 11 Pro (Build 22631)	`192.168.0.116`	Vulnerable FTP (:21), HTTP (:80), and dynamic Windows Firewall quarantine agent.
Secondary Victim	Apple iPadOS 17 (Darwin BSD)	`192.168.0.152`	VLC HTTP Sharing (:80); evaluates Darwin option sequence (WS < TS) and initial TTL 64.
Mobile Target	Android 14 (Linux Kernel 5.x)	`192.168.0.169`	WiFi FTP Server (:2221); evaluates Linux option layout, RFC 6864 IP ID = 0, and closed RST.
Network Appliance	Embedded Linux (Gateway Router)	`192.168.0.1`	DNS (:53) and HTTP Gateway (:80); evaluates embedded appliance TTL and window defaults.

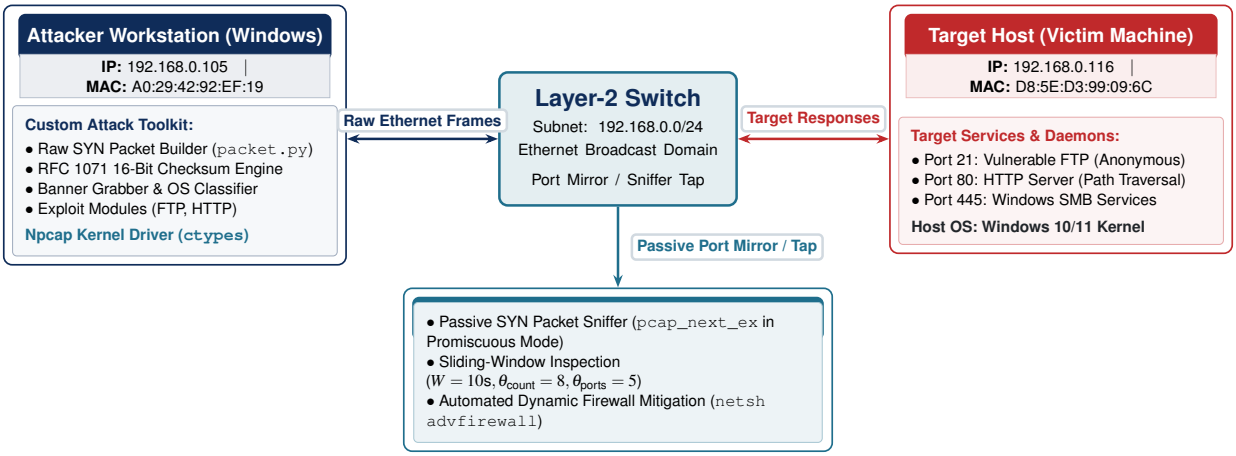


Figure 1: Laboratory Network Topology, Host Roles, and Traffic Monitoring Architecture

2.2 Component Specifications

Table 3: System Component Inventory and Operational Responsibilities

Component	Host Machine	Software Basis	Role in Evaluation
SYN Scanner	Attacker	Python / `packet.py`	Builds raw Ethernet/IPv4/TCP frames, dispatches SYN packets, and logs responses.
Banner Grabber	Attacker	Python / `banner.py`	Performs stream handshakes to capture service banners from open ports.
OS Classifier	Attacker	Python / `fingerprint.py`	Parses SYN/ACK attributes to calculate operating system probability scores.
Exploit Suite	Attacker	Python / `exploits/`	Executes targeted exploit modules against confirmed vulnerable ports.
Npcap Driver	Attacker/Victim	Npcap Native DLLs	Provides low-level kernel packet capture and raw transmission interfaces.
Vulnerable FTP	Victim	Python / `ftp_server.py`	Simulates standard RFC 959 FTP with anonymous read authorization.
Vulnerable HTTP	Victim	Python / `http_server.py`	Web server with intentional path traversal vulnerability (CVE-2021-41773 analogue).
Defense Monitor	Victim	Python / `detector.py`	Passive traffic sniffer calculating sliding-window SYN probe arrival rates.
Firewall Agent	Victim	Windows `netsh`	Injects dynamic inbound blocking rules when scanning thresholds are breached.

2.3 Link-Layer Hardware Address Resolution Architecture

Constructing raw Layer-2 Ethernet II frames requires both the source MAC address of the local network interface and the destination MAC address of the target host. Because standard Python socket libraries on Windows do not expose link-layer address tables directly, the system queries the Windows IP Helper library (`iphlpapi.dll`) through `ctypes`:

1. **Local Interface and Source MAC Discovery:** The scanner identifies the active network interface by opening an ephemeral UDP probe to the target subnet and reading the bound local IP address. It then calls `GetAdaptersInfo()` to extract the 6-octet hardware address of that interface.
2. **Pre-Flight Target ARP Resolution:** Before probe dispatch, the toolkit issues an ARP request using `SendARP(dst_ip, src_ip, pMac, &len)`. This populates the local ARP cache and retrieves the target MAC address over the local broadcast segment.

This design allows the Layer-2 frame construction pipeline (`build_ethernet()`) to operate without hardcoded hardware addresses.

3 Protocol Mechanics and Attack Timing Diagrams

3.1 Original Protocol vs. SYN Port Scan Mechanics

Standard TCP (RFC 793) connections complete a full three-way handshake: SYN → SYN/ACK → ACK. Application listeners accept and log the connection only after the final ACK arrives.

In contrast, our TCP SYN scanner performs a half-open scan. It sends a single SYN probe and inspects the reply. When a port is open, the target returns a SYN/ACK. The scanner records the open state and withholds the final ACK, leaving the connection unestablished and unlogged by application listeners. When a port is closed, the target kernel returns a RST packet. If a firewall drops the probe, the scanner waits until the 1.5-second timeout expires.

3.1.1 Dual-Candidate Ingestion Flow: Closed-Port RST Fallback Pipeline

Passive fingerprinters typically depend on capturing an open-port SYN/ACK frame. When all probed ports on a target are closed, conventional tools fail to classify the operating system.

Our design addresses this with a dual-candidate telemetry pipeline:

- **Primary Candidate (Open Port SYN/ACK):** When a probed port responds with SYN = 1 and ACK = 1, the frame is selected as the primary fingerprint candidate to examine negotiated options, window scale factors, and timestamp fields.
- **Fallback Candidate (Closed Port RST):** When a probed port responds with RST = 1, the scanner extracts the IPv4 TTL, TCP Window Size, and RFC 6864 IP ID. If no open port is found across all probes, the classifier uses the RST packet as fallback telemetry.

An inbound RST frame with Initial TTL = 128 and Window = 0 indicates a Microsoft Windows stack, while an RST with Initial TTL = 64 and IP ID = 0 indicates a Linux kernel. This allows the scanner to identify the operating system family even when every probed port is closed.

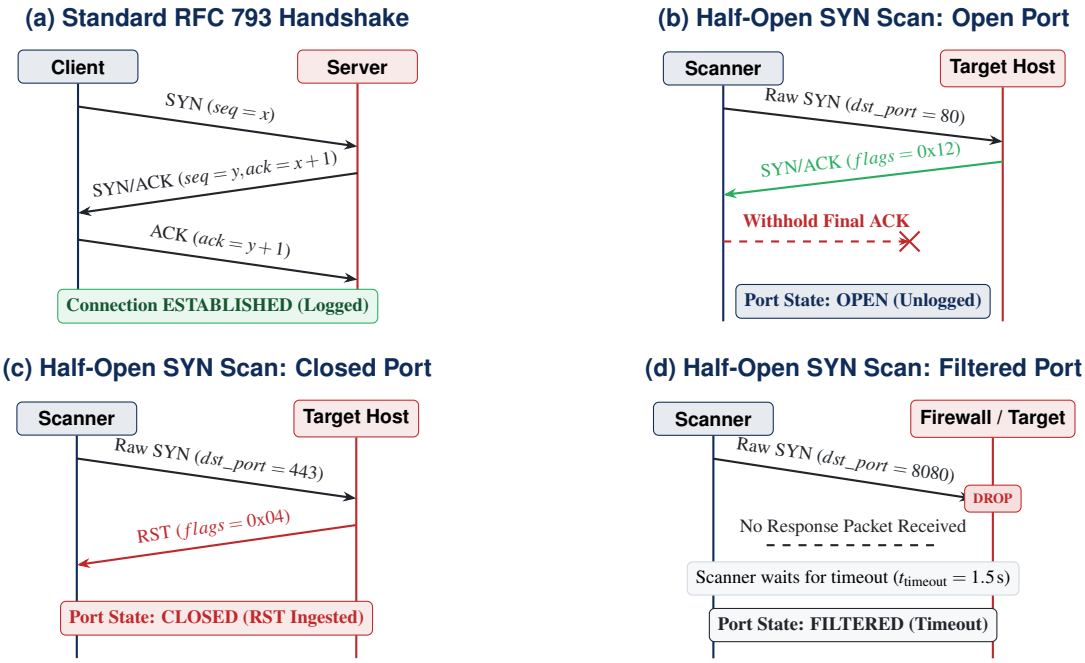


Figure 2: Protocol Timing Diagrams Comparing Standard TCP Handshake with Scanner Probe Classifications

3.2 Service Banner Grabbing Timing Sequences

Banner grabbing extracts identification strings from services running on open ports. As shown in Figure 3, banner grabbing follows two patterns depending on whether the target service is passive or active:

- **Passive Services (e.g., FTP, SMTP):** The listening daemon transmits an informational greeting banner immediately upon completion of the initial TCP 3-way handshake. The grabber establishes a standard connection, reads the incoming buffer, and terminates cleanly.

- **Active Services (HTTP):** Web daemons do not output unprompted greetings. The grabber establishes a connection, issues a minimal RFC-compliant probe (`HEAD / HTTP/1.0\r\n\r\n`), and extracts the software identification from the HTTP `Server:` header.

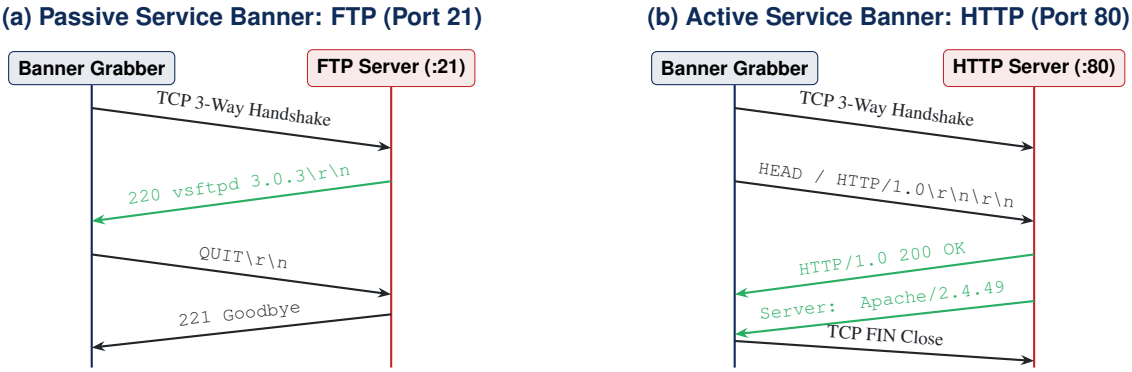


Figure 3: Timing Sequences for Passive Banner Capture and Active Application Probing

3.3 Targeted Exploit Execution Timing

After cataloging open ports and daemon versions, the orchestrator runs the targeted exploit modules. Figure 4 outlines the message sequences for anonymous FTP data access and HTTP path traversal.

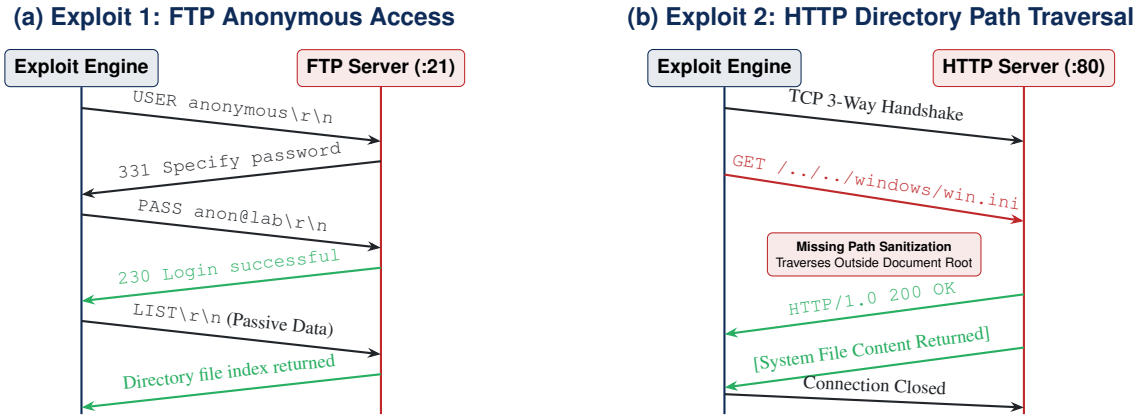


Figure 4: Protocol Interactions Demonstrating Concrete Exploit Outcomes

3.4 Defense Detection and Mitigation Timeline

Figure 5 shows the defense monitor timeline. Incoming SYN packets update a sliding-window queue on the target host. When probe volume reaches 8 SYN packets across at least 5 distinct ports within 10 seconds, the monitor triggers an alert and adds an inbound firewall rule blocking the scanner's IP. The network interface drops subsequent probes before they reach application sockets.

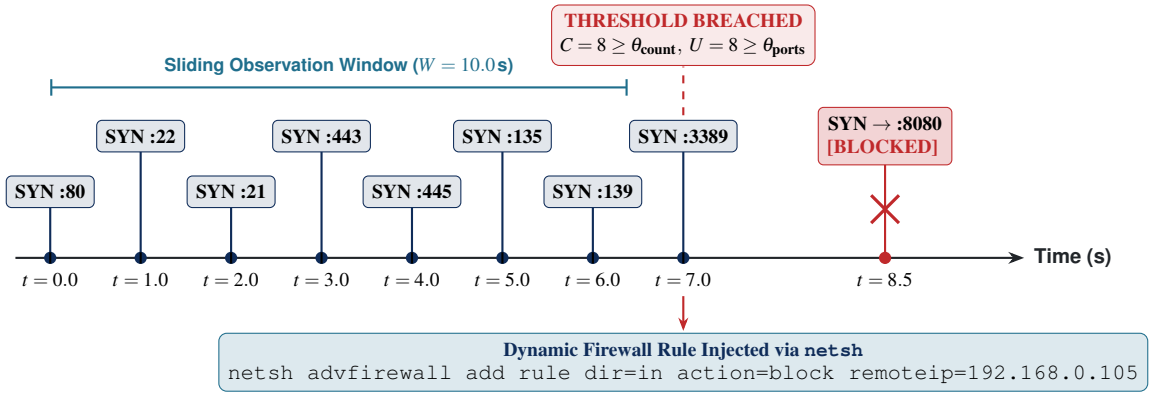


Figure 5: Timeline of Sliding-Window Scan Detection and Dynamic Firewall Rule Injection

4 Binary Packet and Frame Structure

4.1 Layer 2: Ethernet II Frame Header

Every transmitted probe is encapsulated in an Ethernet II frame consisting of 14 header octets:



Figure 6: Layer 2 Ethernet II Frame Structure (Total Header: 14 Octets)

4.2 Layer 3: IPv4 Packet Header

The IPv4 packet is constructed strictly according to RFC 791. Standard probes use a 20-byte base header without IP options:

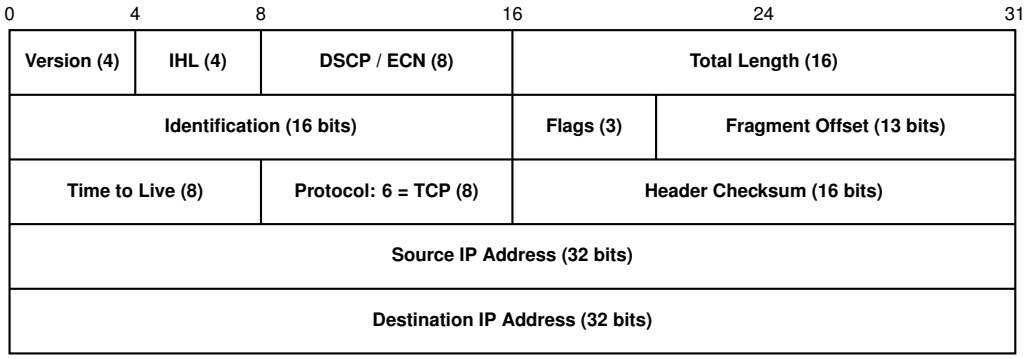


Figure 7: Layer 3 IPv4 Packet Header Format (20 Octets Base)

4.3 Layer 4: TCP Segment Header and Active Option Offering

The TCP header is serialized per RFC 793 with control flags explicitly set for the SYN half-open probe ($Flags = 0x02$). While a minimal TCP header spans 20 octets (Data Offset = 5), standard probes in our proposed toolkit are structured as 40-octet frames (Data Offset = 10) incorporating 20 octets of diagnostic TCP options (`STANDARD_SYN_OPTIONS`):

- **Maximum Segment Size (MSS):** Option Kind 2, Length 4, Value 1460 (`0x02 0x04 0x05 0xb4`).
- **SACK Permitted:** Option Kind 4, Length 2 (`0x04 0x02`), advertising RFC 2018 selective acknowledgment support.
- **TCP Timestamp:** Option Kind 8, Length 10 (`0x08 0x0a ...`), advertising RFC 7323 timestamp capabilities.
- **Window Scale:** Option Kind 3, Length 3, Shift count 8 (`0x03 0x03 0x08`), padded with a NOP (`0x01`) for 32-bit boundary alignment.

Offering these options prompts the target host to negotiate and reveal its native TCP option order, window scale factor, and timestamp support in the SYN/ACK reply.

4.3.1 Active Stimulus-Response TCP Option Formulation

Under RFC 7323 (TCP High Performance Extensions) and RFC 2018 (Selective Acknowledgment), advanced TCP capabilities require mutual negotiation during the initial handshake. If a scanner sends a bare 20-byte TCP SYN header without options, target operating systems omit Window Scale, SACK-Permitted, and Timestamp fields in their SYN/ACK replies.

Our probe design addresses this by including an active option stimulus in every outgoing SYN packet:

$$\text{SYN}_{\text{options}} = \langle \text{MSS}(1460),\ \text{SACK_PERMITTED},\ \text{TIMESTAMP}(t_0, 0),\ \text{NOP},\ \text{WSCALE}(8) \rangle \quad (1)$$

Offering these capabilities prompts the target kernel to return its full set of negotiated parameters:

- The compile-time **option sequence order**, which varies by vendor (for example, Apple Darwin stacks place Window Scale before Timestamp, while Linux stacks place SACK-Permitted before Timestamp).

- The native **Window Scale factor** ($S \in \{5,6,7,8\}$).
- Adherence to RFC 6864, which determines whether the target clears the IPv4 Identification field on atomic datagrams.

The base 20-octet header structure is shown in Figure 8:

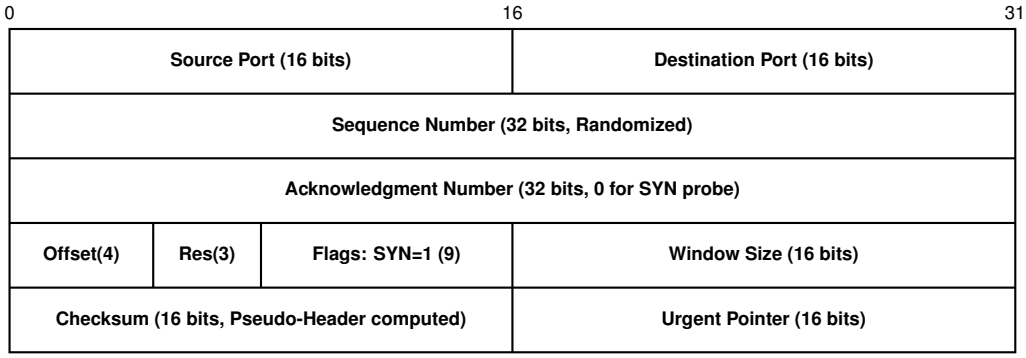


Figure 8: Layer 4 TCP Segment Header Format (20 Octets Base)

4.4 TCP Checksum Calculation and Pseudo-Header Construction

Under RFC 793, the TCP checksum is not calculated over the TCP segment alone. It covers a 12-byte pseudo-header prepended to the TCP segment to verify that the packet reached its intended destination IP address:

$$\text{Checksum} = \sim \left(\sum_{i=1}^{N} W_i \pmod{2^{16}-1} \right) \tag{2}$$

where W_i represents consecutive 16-bit words across the concatenated pseudo-header and TCP segment. The pseudo-header structure is defined in Table 4:

Table 4: IPv4 TCP Pseudo-Header Field Layout for Checksum Calculation

Byte Offset	Field Name	Value in SYN Probe
0–3	Source IPv4 Address	Attacker IPv4 (e.g., `192.168.0.105`)
4–7	Destination IPv4 Address	Target IPv4 (e.g., `192.168.0.116`)
8	Reserved Zero Octet	`0x00`
9	Protocol Identifier	`0x06` (TCP Transport Protocol)
10–11	TCP Segment Length	Length of TCP header + options + payload (e.g., 20)

4.5 OS Fingerprinting Diagnostic Signature Matrix

The fingerprinting engine parses incoming response frames (open-port SYN/ACKs or closed-port RSTs) and extracts diagnostic attributes across Layer 3 and Layer 4 headers. Table 5 defines the baseline signature matrix across target operating system families:

4.6 Mathematical Formulation of Probabilistic OS Scoring and Version Tiering

Passive operating system detection cannot rely on simple exact matches, because middleboxes and overlapping TCP window configurations often obscure single fields. The proposed engine formulates OS identification as a weighted evidence score across four classes: $\mathscr{C} = \{\text{Windows}, \text{Linux}, \text{macOS / BSD}, \text{Network device}\}$.

4.6.1 Evidence-Based Scoring Model

For an extracted feature set $F = \{\text{TTL}_{\text{obs}}, W_{\text{obs}}, \text{DF}, \text{ID}_{\text{obs}}, \text{Options}_{\text{order}}, \text{IsRST}\}$, the score S_c for each class $c \in \mathscr{C}$ is computed as:

$$S_c = \sum_k w_{k,c} \cdot \phi_k(F, c) \tag{3}$$

Table 5: TCP/IP Stack Fingerprinting Signatures across Operating System Families

Diagnostic Feature	Protocol Field	Windows 10/11	Linux Modern	macOS / Modern BSD	Network Device
Initial TTL (SYN/ACK)	IPv4 TTL	128	64	64	255
Don't Fragment (DF)	IPv4 Flags	Set	Set	Set	Often Clear
IPv4 ID (RFC 6864)	IPv4 ID	Non-Zero / Random	Strictly 0 (with DF)	Non-Zero / Sequential	Sequential
Advertised Window	TCP Window	64240, 65535, 8192	29200, 65160, 14600	65535, 33304	4128, 8192
Window Scale Factor	TCP Option 3	8	7 or 8	5 or 6	0 or 2
TCP Timestamp	TCP Option 8	Often Absent	Present	Present	Typically Absent
Option Sequence	TCP Options	M, W, S, T or M, W, S	M, S, T, W (SACK < TS)	M, W, T, S (WS < TS)	MSS Only / None
Closed-Port RST TTL	IPv4 TTL (RST)	128	64	64	255
Closed-Port Window	TCP Window (RST)	0	0	0	0
Closed-Port RST IP ID	IPv4 ID (RST)	Non-Zero	0 (with DF set)	Non-Zero / DF Clear	Non-Zero

where $\phi_k(F,c) \in \{0,1\}$ indicates whether observed feature k satisfies the signature predicate for class c, and $w_{k,c} > 0$ represents diagnostic confidence weightings:

- **Normalized Initial TTL** ($w = +4.0$)**:** Observed TTL T_{obs} is normalized to standard boundary $T_{\text{norm}} \in \{32, 64, 128, 255\}$ via $T_{\text{norm}} = \min\{t \in \{32, 64, 128, 255\} \mid t \geq T_{\text{obs}}\}$. Estimated router hops are $H = T_{\text{norm}} - T_{\text{obs}}$.
- **TCP Option Sequence Order** ($w = +3.0$)**:** Option sequence order separates Linux from Darwin directly: Linux kernels place SACK-Permitted before Timestamp ($\langle \text{MSS}, \text{SACK}, \text{TS}, \text{WS} \rangle$), whereas Apple Darwin and BSD stacks place Window Scale before Timestamp ($\langle \text{MSS}, \text{WS}, \text{TS}, \text{SACK} \rangle$).
- **RFC 6864 IPv4 Identification** ($w = +1.5$ / $+0.8$)**:** When DF is asserted, $\text{ID}_{\text{obs}} = 0$ provides $+1.5$ weight to Linux; non-zero ID awards $+0.8$ to Windows and $+0.4$ to macOS/BSD.
- **Closed-Port RST Ingestion** ($w = +4.5$)**:** When zero ports are open, a returned RST with $T_{\text{norm}} = 128$ and $W_{\text{obs}} = 0$ awards $+4.5$ to Windows; $T_{\text{norm}} = 64$ awards $+3.0$ to Linux/BSD.
- **Advertised Window and Window Scale** ($w = +1.5$ / $+1.0$)**:** Evaluates advertised window buffer presets and scale shift exponents against known vendor defaults.

The cumulative raw scores are normalized into a ranked probability distribution vector **P**:

$$P(c) = \frac{\max(0, S_c)}{\sum_{j \in \mathscr{C}} \max(0, S_j)} \times 100\%, \quad \forall c \in \mathscr{C} \tag{4}$$

4.6.2 Hierarchical Version Tiering Model

Because releases within the same operating system family share identical stack parameters, the design classifies targets into coarse family tiers using secondary feature checks:

- **Windows Tiers:** Evaluates initial TTL 32 vs 128, timestamp absence, and legacy buffer sizes (8192 vs 65535) to discriminate between *Windows 10 / 11 / Server 2016+*, *Windows 7 / 8 / Vista*, and *Legacy Windows 9x / 2000*.
- **Linux Tiers:** Differentiates *Linux modern kernel (3.x / 4.x / 5.x+)* from *Linux legacy (2.4 / 2.6)* based on RFC 6864 IP ID zeroing and modern window scaling presets.
- **macOS / BSD Tiers:** Classifies into *macOS / modern BSD* (evaluating Darwin option ordering and window scale 5–6) versus *Legacy BSD / NetBSD*.

5 Implementation Plan and Modular Architecture

5.1 Modular Repository Architecture

The project codebase is organized into four decoupled packages:

- **`common/`:** Shared core networking utilities.
  - `packet.py`: Implements `build_syn_frame()`, `parse_ipv4_tcp_frame()`, and the RFC 1071 checksum routine.
  - `npcap.py`: Provides native `ctypes` wrappers for `wpcap.dll` functions, including `pcap_open`, `pcap_sendpacket`, and `pcap_next_ex`.
- **`attacker/`:** Multi-phase attack orchestration suite.
  - `main.py`: Master CLI coordinator linking all reconnaissance and exploit phases.
  - `scanner.py`: TCP SYN port scanner implementing the half-open probe loop.
  - `banner.py`: Stream socket connection routines for passive and active banner extraction.
  - `fingerprint.py`: Heuristic scorer comparing SYN/ACK attributes against known signatures.
  - `exploits/`: Dedicated modules for anonymous FTP authentication and HTTP directory path traversal.
- **`victim/`:** Reproducible demonstration targets.
  - `ftp_server.py`: Multithreaded Python FTP server configured to accept anonymous logins.
  - `http_server.py`: HTTP daemon containing an intentional path traversal flaw.
- **`defense/`:** Countermeasure implementation.
  - `detector.py`: In-memory sliding-window packet queue tracking per-IP probe frequency and destination port counts.
  - `main.py`: Automated alert dispatcher executing `netsh advfirewall` commands to enforce temporary IP blocks.

5.2 Automated Verification and Unit Testing Framework

To verify packet construction and scoring algorithms before physical network testing, the implementation plan includes an automated unit test suite built with `pytest`. The suite provides test coverage across two dedicated modules:

- `tests/test_packet.py`:
  - **RFC 1071 Checksum Verification:** Validates 16-bit one's complement addition over odd-length and even-length buffers against standard test vectors.
  - **Binary Framing Roundtrip:** Confirms that serialized Ethernet, IPv4, and TCP frames unpack into their original field values without distortion.
  - **TCP Option Parsing:** Tests parsing of option streams containing MSS, Window Scale, SACK-Permitted, and Timestamp fields with variable NOP padding.
- `tests/test_fingerprint.py`:
  - **TTL Normalization and Hop Estimation:** Verifies rounding to standard boundaries (32, 64, 128, 255) across simulated network hops.
  - **RFC 6864 IP ID Evaluation:** Confirms that datagrams with DF $= 1$ and IP ID $= 0$ correctly increase the Linux classification score.
  - **Closed-Port RST Classification:** Tests that zero-window RST responses classify Windows targets accurately when no open ports exist.
  - **Version Tiering Logic:** Verifies hierarchical scoring between legacy and modern Windows and Linux releases.

6 Justification and Technical Viability

The technical approach relies on four standard protocol behaviors:

1. **Deterministic Kernel Responses (RFC 793 / RFC 9293):** Operating system kernels must reply with RST for packets to closed ports and SYN/ACK for active listening ports. Because these replies come directly from the kernel network stack, the scanner determines port states without establishing an application session, and ingests both SYN/ACK and RST frames for passive OS classification.

2. **Link-Layer Transmission via Npcap:** Standard Windows sockets (`SOCK_RAW`) restrict raw TCP packet construction for unprivileged users. Interfacing directly with the Npcap driver via `ctypes` bypasses transport-layer limits and transmits custom Layer-2 Ethernet frames onto the local wire.
3. **Vendor Divergence in TCP Defaults and Option Ordering:** Operating system vendors configure distinct default values for initial TTL, TCP buffer window size, and option negotiation sequences. Windows defaults to an initial TTL of 128, while Linux and Apple Darwin use 64. In addition, Apple Darwin puts Window Scale before Timestamp ($\langle \text{MSS}, \text{WS}, \text{TS}, \text{SACK} \rangle$), while Linux puts SACK-Permitted before Timestamp ($\langle \text{MSS}, \text{SACK}, \text{TS}, \text{WS} \rangle$). This difference in sequence order distinguishes the two platforms directly.
4. **RFC 6864 IPv4 Identification Semantics:** Modern Linux kernels strictly implement RFC 6864 by zeroing out the IPv4 Identification field (IP ID $= 0$) on all datagrams where the Don't-Fragment (DF) bit is set. Conversely, Windows and BSD-derived kernels maintain non-zero, sequential, or randomized IP ID values regardless of the DF flag. This protocol property reliably disambiguates Linux hosts from macOS/BSD hosts even when both share an identical initial TTL of 64.

6.1 Operational Resilience and Failure Handling

Beyond protocol viability, the system handles operational faults through explicit state tracking:

- **Asynchronous Probe Timeouts:** Network operations enforce a 1.5-second per-probe timeout so that dropped or filtered packets do not stall scanner thread execution.
- **Automated Firewall Rule Cleanup:** Dynamic firewall rules added via `netsh` are tracked in memory and removed upon exit, preventing persistent network lockouts.
- **Sandboxed Demonstration Services:** Target services run inside dedicated unprivileged test directories to keep proof-of-concept verification isolated from host files.

7 Expected Outcomes of a Successful Attack

7.1 Evaluation Criteria

Table 6 summarizes the target criteria for evaluating each attack phase:

Table 6: Evaluation Metrics and Validation Methods for Attack Phases

Phase	Evaluation Target	Success Indicator	Validation Method
1. Port Scan	Accurate port classification	Port states match host reality (Open, Closed, Filtered).	Compare scanner output with `netstat -an` on victim host.
2. Banners	Software identification	Extracted string contains correct daemon name and version.	Match against service launch configurations and headers.
3. OS Fingerprint	OS family and version tier	Target classified with ranked probability distribution ($\geq 60\%$) and version tier identified.	Compare against victim system properties (`winver`, `uname -r`, `sw_vers`).
4. Exploits	Access compromise	Unauthorized read/access on target services demonstrated.	Session logs confirm anonymous FTP authentication and HTTP directory path traversal read.

7.2 Anticipated Execution Telemetry and Verification Output

To verify pipeline behavior against its functional design criteria, the anticipated console output of an automated reconnaissance and exploit cycle is shown below. This trace details the data schema, probabilistic classifications, and exploit verification results:

Attacker Console Output: Benchmark Execution Trace

```
============================================================
  CUSTOM PORT SCANNER: ATTACK TOOLKIT
============================================================
  Target       : 192.168.0.116
  Attacker     : 192.168.0.105 (a0:29:42:92:ef:19)
  Target MAC   : d8:5e:d3:99:09:6c
  Ports        : [21, 22, 80, 81]
  Scan delay   : 0.5s

  [PHASE 1] TCP SYN Port Scanning
  ------------------------------------------------------------
     21  OPEN                            TTL=128 WIN=65535
     22  CLOSED                          TTL=128 WIN=0
     80  OPEN                            TTL=128 WIN=65535
     81  CLOSED                          TTL=128 WIN=0

  [PHASE 2] Service Banner Grabbing
  ------------------------------------------------------------
  [+]  21  FTP   -> LabFTP Server (Vulnerable Demo) Ready
  [+]  80  HTTP  -> VulnHTTP/1.0 (Lab Demo)

  [PHASE 3] OS Fingerprinting
  ------------------------------------------------------------
  Candidate  : Open-Port TCP SYN/ACK
  Likely OS  : Windows
  Confidence : 60%
  Version    : Windows 10 / 11 / Server 2016+
               71%  Windows 10 / 11 / Server 2016+
               29%  Windows 7 / 8 / Vista / Server 2008-2012
                0%  Windows 9x / 2000 (legacy)
  TTL        : 128 (initial 128, ~0 hop(s))
  Window     : 65535
  DF bit     : set
  IP ID      : 60211
  Options    : ('MSS', 1460), ('NOP',), ('WSCALE', 8),
               ('SACK_PERMITTED',), ('TIMESTAMP', ...)
  Probability:
    Windows           60%  ############
    macOS / BSD       21%  ####
    Linux             19%  ###
    Network device     0%
  Evidence   :
    - initial TTL 128 (~128 observed, 0 hop(s)) matches Windows
    - TCP window 65535 matches Windows, Linux, macOS / BSD
    - Don't-Fragment (DF) bit set - modern TCP stack
    - TCP option order ['MSS', 'WSCALE', 'SACK_PERMITTED', 'TIMESTAMP']
      matches Windows layout
    - IP ID is non-zero (60211) with DF set (Windows/BSD characteristic)

  [PHASE 5] Targeted Exploit Execution
  ------------------------------------------------------------
  [EXPLOIT 1] FTP Anonymous Authentication (Port 21)
  Status     : SUCCESS (Authenticated as anonymous:guest)
  Directory  : 257 "/" | Files: drwxr-xr-x 1 ftp ftp 0 pub

  [EXPLOIT 2] HTTP Directory Path Traversal (Port 80)
```

```
  Status     : SUCCESS (Payload: ..\..\..\windows\win.ini)
  Payloads   : 12 attempted, 8 successful variations
  Disclosed  : [fonts] Arial=arial.ttf (arbitrary host file read)

============================================================
  ATTACK SUMMARY
============================================================
  Ports scanned : 4 | Open: 2 | Closed: 2
  Banners found : 2/2 | OS guess: Windows (60%)
  Exploits      : 2/2 successful (FTP Anonymous + HTTP Traversal)
============================================================
```

When active mitigation is enabled, the defense monitor on the victim detects the probe frequency and applies a dynamic firewall rule. The resulting console telemetry demonstrates the mitigation effect:

Victim Console Output: Defense Detection and Firewall Quarantine

```
[14:20:05.12] [MONITOR] Inbound SYN probe received on port 21 from 192.168.0.105
[14:20:05.63] [MONITOR] Inbound SYN probe received on port 22 from 192.168.0.105
[14:20:06.14] [MONITOR] Inbound SYN probe received on port 80 from 192.168.0.105
[14:20:06.65] [MONITOR] Inbound SYN probe received on port 81 from 192.168.0.105
[14:20:07.16] [MONITOR] Inbound SYN probe received on port 135 from 192.168.0.105
[14:20:07.67] [MONITOR] Inbound SYN probe received on port 139 from 192.168.0.105
[14:20:08.18] [MONITOR] Inbound SYN probe received on port 443 from 192.168.0.105
[14:20:08.69] [MONITOR] Inbound SYN probe received on port 445 from 192.168.0.105
[14:20:08.70] [ALERT] Port scan detected from 192.168.0.105! (8 SYNs in 3.6s)
[14:20:08.72] [MITIGATION] Adding Firewall rule: CustomPortScanBlock_192_168_0_105
[14:20:08.78] [MITIGATION] Inbound TCP traffic from 192.168.0.105 is now BLOCKED.
```

Following rule insertion, subsequent scanner probes to the victim receive no reply, changing the port state to `NO RESPONSE / POSSIBLY FILTERED` and preventing exploit connections.

8 Defense Mechanism and Dynamic Countermeasures

8.1 Sliding-Window SYN Rate and Diversity Detection

Port scanning generates a characteristic footprint: multiple SYN packets directed at different destination ports within a short interval. The defensive detector monitors incoming TCP traffic using a two-part sliding window:

Detection Algorithm Formulation

For each unique source IP S, maintain a sliding event queue:

$$Q_S = \{(t_i, p_i) \mid t_i > t_{\text{current}} - W\}$$

where W is the sliding window duration ($W = 10.0$ seconds). On arrival of any packet with SYN $= 1$ and ACK $= 0$:

1. Append $(t_{\text{current}}, p_{\text{dest}})$ to Q_S.
2. Evict all entries where $t_i \leq t_{\text{current}} - W$.
3. Compute total events $C = |Q_S|$ and unique ports $U = |\{p_i \mid (\cdot, p_i) \in Q_S\}|$.
4. Trigger alert if $C \geq \theta_{\text{count}}$ (8) **and** $U \geq \theta_{\text{ports}}$ (5).

Checking both packet rate and port diversity prevents false positives from single-port bursts (such as parallel HTTP asset downloads) while catching multi-port scans.

8.2 Dynamic Host Firewall Mitigation

When the monitor triggers an alert and mitigation is enabled, it adds an inbound blocking rule through the Windows Advanced Firewall:

Windows Advanced Firewall Dynamic Block Rule Command

```
netsh advfirewall firewall add rule name="CustomPortScanBlock_192_168_0_105" ^
    dir=in action=block remoteip=192.168.0.105 protocol=TCP profile=any
```

The defense process records active rules in memory and removes them upon exit to prevent persistent network misconfiguration.

8.3 Security Analysis and Limitations

- **False-Positive Suppression:** Requiring destination port diversity distinguishes scans from high-throughput single-port traffic, such as web browsing or database connection pools.
- **Sub-Second Mitigation:** Rule generation via `netsh` applies within milliseconds of crossing the detection threshold.
- **Known Limitations:** Slow scans with probe intervals longer than the 10-second window will evade detection. Attackers could also bypass the single-IP filter by distributing probes across multiple addresses.

References

[1] Postel, J., "Transmission Control Protocol," *RFC 793*, Defense Advanced Research Projects Agency, 1981.
[2] Postel, J., "Internet Protocol," *RFC 791*, Defense Advanced Research Projects Agency, 1981.
[3] Braden, R., Borman, D., and Partridge, C., "Computing the Internet Checksum," *RFC 1071*, 1988.
[4] Touch, J., "Updated Specification of the IPv4 ID Field," *RFC 6864*, 2013.
[5] Borman, D., Braden, B., Jacobson, V., and Scheffenegger, R., "TCP Extensions for High Performance," *RFC 7323*, 2014.
[6] Mathis, M., Mahdavi, J., Floyd, S., and Romanow, A., "TCP Selective Acknowledgement Options," *RFC 2018*, 1996.
[7] Eddy, W., "Transmission Control Protocol (TCP)," *RFC 9293*, 2022.
[8] Fyodor (Lyon, G.), "Remote OS Detection via TCP/IP Stack Fingerprinting," *Phrack Magazine*, vol. 8, no. 54, 1998.
[9] MITRE ATT&CK, "Network Service Scanning (T1046)," *MITRE Corporation*, 2023. https://attack.mitre.org/techniques/T1046/.
[10] National Vulnerability Database, "CVE-2021-41773: Apache HTTP Server Path Traversal Vulnerability," *NIST*, 2021. https://nvd.nist.gov/vuln/detail/CVE-2021-41773.
[11] Nmap Project, "Npcap: Windows Packet Capture Library & Driver," *Npcap Reference Documentation*, 2024. https://npcap.com/.